



Módulo 2

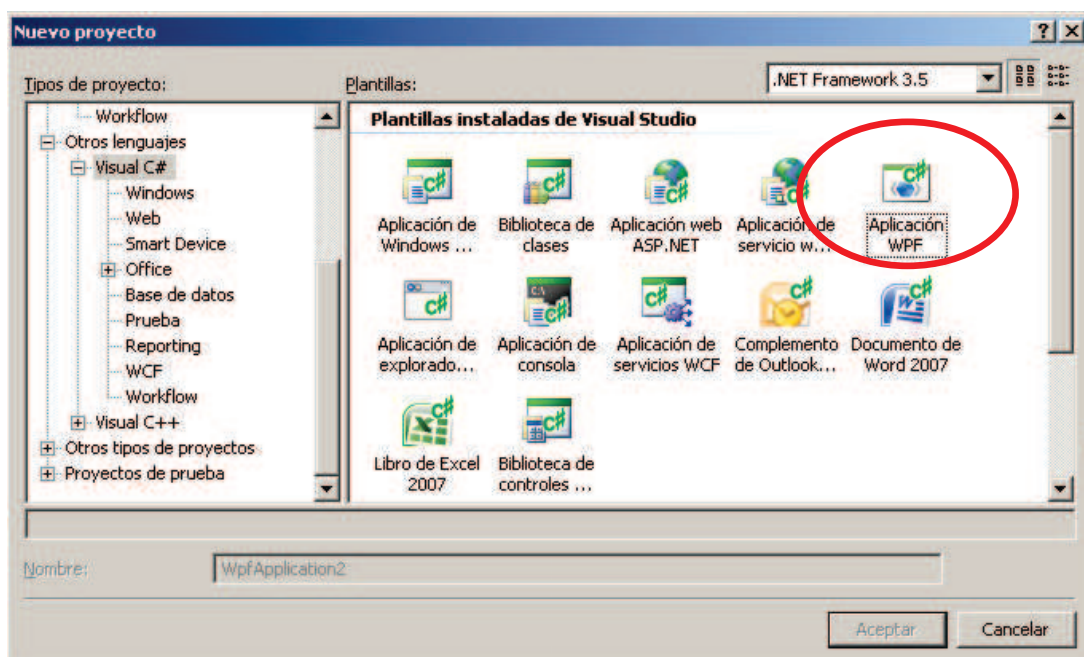
Creando Aplicaciones WPF

1- Crear una aplicación WPF

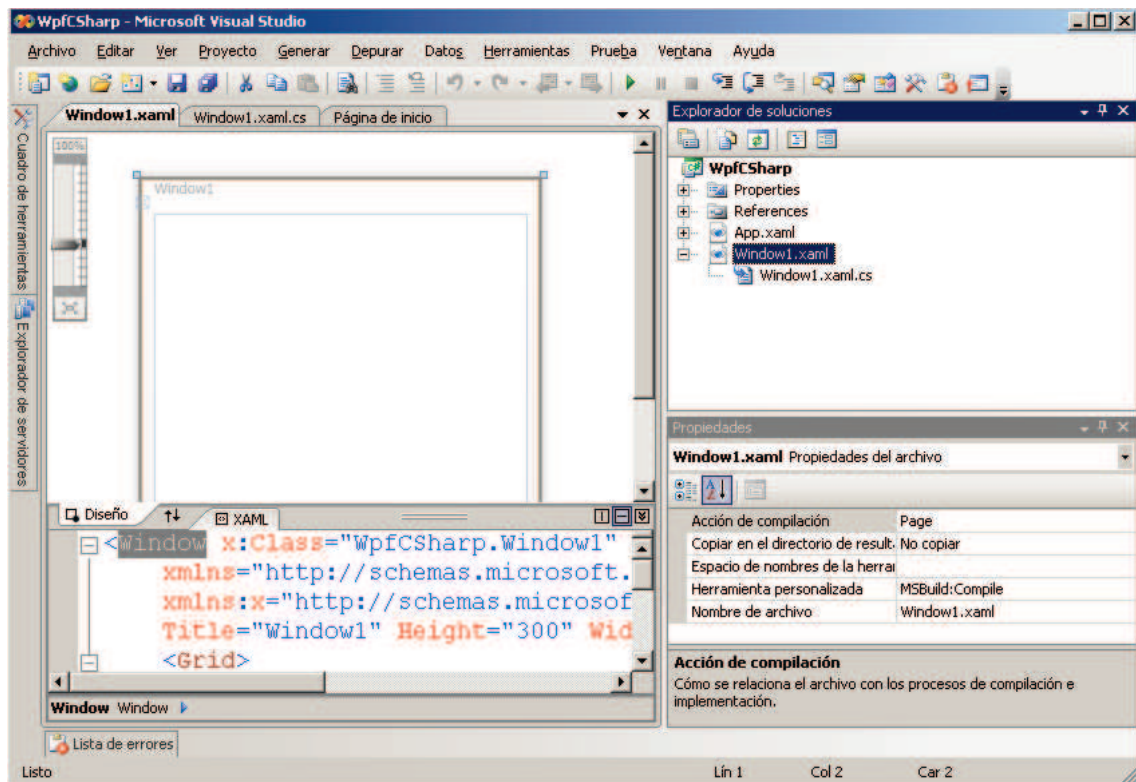
Aplicación WPF

Dentro de Visual Studio:

- En el menú *Archivo*, haga clic en *Nuevo proyecto*. Aparecerá el cuadro de diálogo de *Nuevo proyecto*.
- Seleccione el lenguaje y elija **Aplicación WPF**.
- Cambie el nombre de la aplicación.
- Haga clic en *Aceptar*.



Se creará una nueva carpeta para el proyecto con el nombre del proyecto y, a continuación, mostrará el nuevo formulario WPF titulado Window1 en la vista Diseñador. Dentro de esta vista se puede trabajar el formulario de forma similar al de las aplicaciones Windows. Arrastre controles y modifique sus propiedades usando la ventana de propiedades. Escriba los atajadores de eventos en el archivo .xaml.cs o .xaml.vb según el lenguaje seleccionado. La ventana de WPF que se ve en la vista Diseñador es una representación visual de la ventana que se abrirá al iniciar la aplicación. En la vista Diseñador, puede arrastrar diversos controles desde el Cuadro de herramientas hasta la ventana WPF. Después de haber colocado un control a la ventana de WPF, Visual C# crea automáticamente código que hará que el control se sitúe apropiadamente cuando se ejecute el programa.



Clase Window

Permite crear, configurar, mostrar y administrar la duración de las ventanas y los cuadros de diálogo. El punto de interacción entre el usuario y una aplicación independiente es una ventana. Una ventana de WPF se compone de dos áreas distintas:

- Un área no cliente, que hospeda los elementos gráficos de las ventanas, incluidos un icono, título, menú del sistema, botón para minimizar, botón para maximizar, botón para restablecer, botón para cerrar y un borde.
- Un área cliente, que hospeda el contenido específico de la aplicación.

```
<Window x:Class="ApIWPf.Principal"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
Title="Principal" Height="500" Width="500" WindowStyle="ThreeDBorderWindow"
ResizeMode="CanResize" ShowInTaskbar="True" WindowState="Maximized">
```

Configurar la ventana de inicio

El archivo **App.xaml** y el archivo de código subyacente correspondiente se crean de forma predeterminada en un proyecto WPF. Este archivo contiene recursos de nivel de aplicación que pueden ser usados en cualquier documento de la aplicación. Dentro de este archivo se implementa la clase **Application**. La ventana de inicio se establece a través de la propiedad **StartupUri**.

```
<Application x:Class="WpfCSharp.App"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
StartupUri="Principal.xaml">
```



<Application.Resources>

</Application.Resources>

</Application>

Propiedades de la Clase Window

AllowsTransparency: Obtiene o establece un valor que indica si se puede ajustar la opacidad del formulario

Background: Obtiene o establece un pincel (Brush) que describe el fondo de un control.

BorderBrush: Obtiene o establece un pincel (Brush) que describe el fondo del borde de un control.

BorderThickness: Obtiene o establece el grosor del borde de un control.

Cursor: Obtiene o establece el cursor que se muestra cuando el puntero del mouse se sitúa sobre este elemento.

Foreground: Obtiene o establece un pincel (Brush) que describe el color de primer plano, generalmente la letra.

Height: Obtiene o establece el alto sugerido del elemento, expresados en unidades independientes de dispositivo (1/96 de pulgada por unidad).

Icon: Obtiene o establece el icono de una ventana.

IsEnabled: Obtiene o establece si la ventana permite recibir entradas del usuario.

Left: Obtiene o establece la posición del borde izquierdo de la ventana con respecto al escritorio.

ResizeMode: Obtiene o establece el modo de cambio de tamaño.

ShowInTaskBar: Obtiene o establece un valor que indica si la ventana tiene un botón de barra de tareas.

SizeToContent: Obtiene o establece un valor que indica si una ventana ajustará automáticamente su tamaño al de su contenido.

Title: Obtiene o establece el título de una ventana.

Top: Obtiene o establece la posición del borde superior de la ventana con respecto al escritorio.

TopMost: Obtiene o establece un valor que indica si una ventana aparece en el punto más alto en el orden Z.

Width: Obtiene o establece el ancho del elemento, expresados en unidades independientes de dispositivo (1/96 de pulgada por unidad).

WindowStartupLocation: Obtiene o establece la posición de la ventana cuando se muestra por primera vez.

WindowState: Obtiene o establece un valor que indica si una ventana está restaurada, minimizada o maximizada.

WindowStyle: Obtiene o establece el estilo de borde de una ventana.

Controles WPF y su equivalente con Controles Windows Forms

Windows Forms	WPF	Comentarios
Button	Button	
CheckBox	CheckBox	
CheckedListBox	ListBox Compuestos	
ColorDialog		



ComboBox	ComboBox	
ContextMenuStrip	ContextMenu	
DataGridView		Algunas funcionalidades de DataGridView se pueden reproducir usando los controles ListView y GridView.
DateTimePicker		
ErrorProvider		
FlowLayoutPanel	WrapPanel o StackPanel	
FolderBrowserDialog		
FontDialog		
Form	Window	
GroupBox	GroupBox	
HelpProvider		No tiene ayuda con F1. Se puede reemplazar esta funcionalidad con ToolTips
ImageList		
Label	Label	
LinkLabel		Se puede utilizar la clase Hyperlink.
ListBox	ListBox	
ListView	ListView	Solo contiene la vista de detalle en formato de solo.
MaskedTextBox		
MenuStrip	Menu	
MonthCalendar		
NotifyIcon		
NumericUpDown	TextBox y dos RepeatButton	
OpenFileDialog	OpenFileDialog	
PageSetupDialog		
Panel	Canvas	



PictureBox	Image	
PrintDialog	PrintDialog	
PrintDocument		
PrintPreviewControl	DocumentViewer	
PrintPreviewDialog		
ProgressBar	ProgressBar	
RadioButton	RadioButton	
RichTextBox	RichTextBox	
SaveFileDialog	SaveFileDialog	
SoundPlayer	MediaPlayer	
SplitContainer	GridSplitter	
StatusStrip	StatusBar	
TabControl	TabControl	
TableLayoutPanel	Grid	
TextBox	TextBox	
Timer	DispatcherTimer	
ToolStrip	ToolBar	
ToolTip	ToolTip	
TrackBar	Slider	
TreeView	TreeView	
WebBrowser	Frame, System.Windows.Controls.WebBrowser	The Frame control can host HTML pages.

Controladores de Eventos

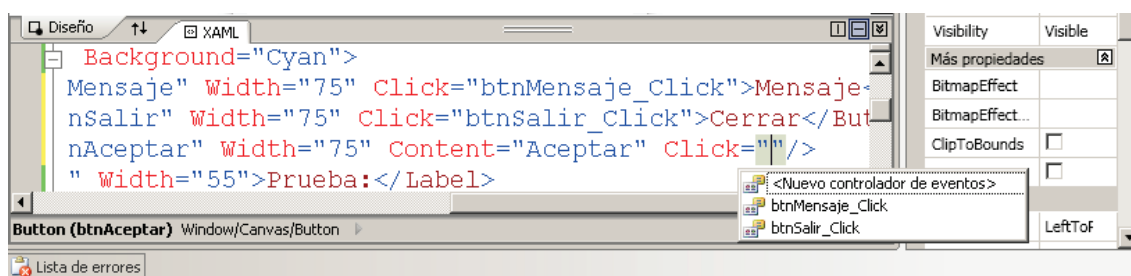
Para crear un controlador de eventos se puede hacer doble clic sobre el control. De esta manera se crea el atajador del evento predeterminado de un control.

También puede agregarse como atributo escribiendo sobre el editor XAML. Al escribir el nombre del evento o seleccionarlo de la lista de ayuda aparecerá una opción <Nuevo



Controlador de Eventos> y el nombre de procedimientos posibles de ser seleccionados teniendo en cuenta la firma del atajador del evento seleccionado. El nombre predeterminado del atajador del evento será NombreObjeto_Evento. El controlador de eventos se escribirá en la clase asociada al formulario.

Todas las librerías del Framework de .Net esta disponible para ser usada en este tipo de aplicaciones.



Recursos

WPF provee recursos como una forma simple de reusar objetos y valores definidos. Se pueden crear recursos en XAML o por código. Todos los elementos de nivel *FrameWork* tienen la propiedad *Resources*, aunque generalmente se los usa en elementos de tipo raíz como *Window*.

Los recursos deben contener una identificación única. Esta se declara usando **x:Key**. Generalmente la clave se la declara como carácter (string), aunque puede ser declarada usando otros tipos de datos.

```
<Page.Resources>
    <SolidColorBrush x:Key="MyBrush" Color="Gold"/>
    <Style TargetType="Border" x:Key="PageBackground">
        <Setter Property="Background" Value="Blue"/>
    </Style>
    <Style TargetType="TextBlock" x:Key="TitleText">
        <Setter Property="Background" Value="Blue"/>
        <Setter Property="DockPanel.Dock" Value="Top"/>
        <Setter Property="FontSize" Value="18"/>
        <Setter Property="Foreground" Value="#4E87D4"/>
        <Setter Property="FontFamily" Value="Trebuchet MS"/>
        <Setter Property="Margin" Value="0,40,10,10"/>
    </Style>
    <Style TargetType="TextBlock" x:Key="Label">
        <Setter Property="DockPanel.Dock" Value="Right"/>
        <Setter Property="FontSize" Value="8"/>
        <Setter Property="Foreground" Value="{StaticResource MyBrush}"/>
        <Setter Property="FontFamily" Value="Arial"/>
        <Setter Property="FontWeight" Value="Bold"/>
        <Setter Property="Margin" Value="0,3,10,0"/>
    </Style>
</Page.Resources>
```

Recursos Estáticos y Dinámicos

Los recursos pueden ser referenciados en forma estática o dinámica. Esto se hace usando **StaticResource** o **DynamicResource** y el nombre del recurso que se está referenciando. La



referencia estática procesa la clave buscando el valor en todos los recursos disponibles, esto ocurre durante la carga del proceso. En cambio la referencia dinámica procesa la clave creando una expresión que permanece sin ser evaluada hasta el momento de la ejecución, durante la ejecución evalúa la expresión y devuelve el valor.

Cuando se va a crear una referencia a un recurso, las siguientes consideraciones deberían tenerse en cuenta para elegir cual de las dos opciones va a usarse:

- Donde está creado el recurso, si es un recurso de ventana, de aplicación, de control, etc.
- La funcionalidad de la aplicación
- El comportamiento del recurso

Cuando elegir *StaticResource*:

- La mayoría de los recursos son de ventana o aplicación. Los recursos no son re evaluados durante la ejecución mejorando la performance.
- Se está estableciendo alguna propiedad del objeto en *Freezable*
- Se están creando recursos que serán compilados en una dll aparte.
- Se están definiendo recursos que van a ser usados en Temas
- Se están usando recursos con muchas propiedades de dependencia.

Cuando elegir *DynamicResource*:

- El valor del recurso depende de una condición que solo se conoce durante la ejecución.
- Se están creando o referenciando Temas para un control personalizado
- Se está intentando ajustar el contenido de *ResourceDictionary*
- Cuando se usan recursos compilados
- Cuando se referencia recursos pesados y el uso de ese recurso no es inmediato, o sea no se usa durante la carga de la ventana. Los recursos estáticos se cargan con la carga de la ventana, en cambio los dinámicos cuando se necesitan
- Se aplican recursos a elemento que dependen de elementos padres. Al cambiar el elemento padre es posible que sea necesario cambiar el objeto hijo también.

```
<Button Background="{StaticResource MyBrush}"/>
<Ellipse Fill="{StaticResource MyBrush}"/>
```

2- Controles de Diseño

Sistema de Diseño de Interfase de Usuario

El término diseño describe el proceso de medir y organizar los integrantes de la colección *Children* de un elemento contenedor y, a continuación, dibujarlos en la pantalla. Se trata de un proceso intensivo y, cuanto mayor es la colección *Children*, mayor es el número de cálculos realizados. También se puede incluir mayor complejidad según el comportamiento de diseño definido por el elemento contenedor que posee la colección. Un diseño relativamente sencillo, como Canvas, puede producir un rendimiento excelente si no se requiere un Panel más complejo como Grid.

Cada vez que un elemento secundario cambia su posición, tiene el potencial de desencadenar un nuevo paso del sistema de diseño. Como a tal, es importante entender los eventos que pueden invocar el sistema del diseño, puesto que la invocación innecesaria puede deteriorar el rendimiento de la aplicación.

En su versión más simple, el diseño es un sistema recursivo que conduce a la configuración del tamaño, posición y representación de un elemento en la pantalla. El sistema de diseño completa dos pasos por cada miembro de la colección *Children*: un paso de medida y un paso de organización. Se trata de la serie de eventos que se producen cada vez que se invoca el sistema de diseño.

- Un elemento secundario comienza el proceso de diseño midiendo, en primer lugar, propiedades básicas.
- Se evalúan las propiedades de tamaño, como *Width*, *Height* y *Margin*.
- Se aplica la lógica concreta del contenedor, como la dirección de *Dock* o la orientación (*Orientation*) de apilado.
- El contenido se organiza después de medir todos los elementos secundarios.
- Se dibuja la colección *Children* en la pantalla.
- Se invoca el proceso de nuevo si se agregan *Children* adicionales a la colección.

Al pensar en el diseño de una aplicación en Windows Presentation Foundation (WPF), es importante entender el rectángulo de selección que rodea todos los elementos. Esta abstracción ayuda a comprender el comportamiento del sistema de diseño. Cada elemento utilizado por el sistema de diseño se puede considerar como un rectángulo que se inserta en una partición del diseño. El sistema, calculando el espacio de pantalla disponible, determina el tamaño de ese rectángulo, el tamaño de cualquier restricción, las propiedades específicas del diseño, como el margen y el relleno, y el comportamiento individual del elemento primario. Al procesar estos datos, el sistema puede calcular la posición de todos los elementos secundarios de un contenedor determinado. Es importante recordar que las características de tamaño definidas en el elemento primario (como *Border*) afectan a sus elementos secundarios.

Cuando se representa el contenido de un objeto *Window*, se invoca el sistema del diseño automáticamente. Para mostrar el contenido, el *Content* de la ventana debe definir un panel raíz que sirve para definir un marco que permite organizar los miembros de *Children* en la pantalla.

A continuación se detallan los controles disponibles como paneles o contenedores.

Border

Dibuja un borde, un fondo o ambos alrededor de otro elemento.

En el siguiente ejemplo se muestra un control *Border* que contiene un *CheckBox*.

```
<Border Background="LightBlue"
BorderBrush="Black"
BorderThickness="2"
CornerRadius="50"
Padding="5" Height="36" VerticalAlignment="Top" Margin="100,44,116,0">
  <CheckBox Name="Prueba" IsChecked="True" >Prueba</CheckBox>
</Border>
```

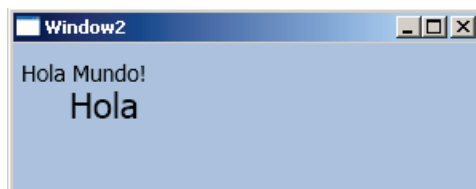


Canvas

Define un área en la que pueden colocarse explícitamente los elementos secundarios utilizando las coordenadas relativas al área del control *Canvas*. *Canvas* es el único elemento del panel que no tiene ninguna característica de diseño inherente. Tiene las propiedades predeterminadas *Height* y *Width* de cero. Siempre se da a los elementos secundarios de *Canvas* el tamaño máximo que requieren. Como resultado, la alineación vertical y horizontal no tiene ningún efecto en un control *Canvas*. *Canvas* es un control de diseño de nivel superior que puede utilizarse para la colocación absoluta del contenido secundario.

En el siguiente ejemplo se muestra un control *Canvas* con dos *TextBlock* contenidos. El uso de las propiedades *Canvas.Top* y *Canvas.Left* permite posicionar los controles contenidos.

```
<Canvas Background="LightSteelBlue">
  <TextBlock FontSize="14" Canvas.Top="10" Canvas.Left="5">Hola
  Mundo!</TextBlock>
  <TextBlock FontSize="22" Canvas.Top="25"
  Canvas.Left="35">Hola</TextBlock>
</Canvas>
```



DockPanel

Define un área en la que se pueden organizar horizontalmente o verticalmente los elementos secundarios, uno respecto al otro. La posición de los elementos secundarios de un control *DockPanel* en la pantalla está determinada por la propiedad *Dock* de los elementos secundarios respectivos y el orden relativo de estos elementos secundarios en *DockPanel*. Por tanto, un conjunto de elementos secundarios con los mismos valores de la propiedad *Dock* se puede organizar de forma diferente en la pantalla dependiendo del orden de estos elementos secundarios dentro del *DockPanel*. La clasificación de los elementos secundarios afecta a la posición porque *DockPanel* recorre en iteración sus elementos secundarios en orden, y establece la posición de cada elemento en función del espacio restante.

El método *SetDock* cambia la posición de un elemento respecto a otros elementos del mismo contenedor. Las propiedades de alineación, como *HorizontalAlignment*, cambian la posición de un elemento respecto a su elemento principal.

Si establece la propiedad *LastChildFill* en *true*, que es el valor predeterminado, el último elemento secundario de *DockPanel* rellena siempre el espacio restante, sin tener en cuenta ningún valor que se haya establecido en este elemento. Para acoplar un elemento secundario en otra dirección, debe establecer la propiedad *LastChildFill* en *false* y especificar además una dirección de acoplamiento explícita para este último elemento secundario.

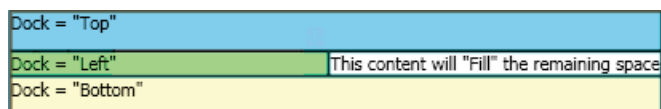
En el siguiente ejemplo se muestra el uso de un control *DockPanel* con varios controles contenidos. La propiedad *DockPanel* permite posicionar los objetos.

```
<DockPanel LastChildFill="True">
  <Border Height="25" Background="SkyBlue" BorderBrush="Black"
  BorderThickness="1" DockPanel.Dock="Top">
    <TextBlock Foreground="Black">Dock = "Top"</TextBlock>
  </Border>
  <Border Height="25" Background="LemonChiffon" BorderBrush="Black"
  BorderThickness="1" DockPanel.Dock="Bottom">
    <TextBlock Foreground="Black">Dock = "Bottom"</TextBlock>
  </Border>
  <Border Width="200" Background="PaleGreen" BorderBrush="Black"
  BorderThickness="1" DockPanel.Dock="Left">
    <TextBlock Foreground="Black">Dock = "Left"</TextBlock>
  </Border>
  <Border Background="White" BorderBrush="Black" BorderThickness="1">
```

```

        <TextBlock Foreground="Black">This content will "Fill" the remaining
        space</TextBlock>
    </Border>
</DockPanel>

```



Grid

Define un área de cuadrícula flexible que está compuesta de columnas y filas. Los elementos secundarios de *Grid* se dibujan en el orden en que aparecen en el código. En consecuencia, se puede lograr un orden dispuesto en capas (también conocido como orden Z) cuando los elementos comparten las mismas coordenadas.

Grid y *Table* comparten alguna funcionalidad común, pero se puede aplicar cada uno de ellos en escenarios adecuados para hacer un mejor uso de sus características integradas. *Grid* agrega elementos basándose en un índice de fila y columna; *Table* no lo hace. El elemento *Grid* permite la disposición en capas del contenido si puede haber más de un elemento dentro de una sola celda. *Table* no admite la disposición en capas. Los elementos secundarios de un elemento *Grid* se pueden colocar de forma absoluta con respecto a la esquina superior izquierda de los límites de su celda. *Table* no admite esta característica. *Grid* también proporciona un comportamiento de cambio de tamaño más flexible que *Table*.

Grid utiliza el objeto *GridLength* para definir las características de ajuste de tamaño de *RowDefinition* o *ColumnDefinition*.

En el siguiente ejemplo se muestra el uso del control *Grid*. La definición inicial de filas y columnas y el uso de las propiedades *Grid.Row* y *Grid.Column* para asociar los controles contenidos a la celda correspondiente.

```

<Grid VerticalAlignment="Top" HorizontalAlignment="Left" ShowGridLines="True"
Width="250" Height="100" Canvas.Left="80" Canvas.Top="67"
Background="LightYellow" >
    <Grid.ColumnDefinitions>
        <ColumnDefinition />
        <ColumnDefinition />
        <ColumnDefinition />
    </Grid.ColumnDefinitions>
    <Grid.RowDefinitions>
        <RowDefinition />
        <RowDefinition />
        <RowDefinition />
        <RowDefinition />
    </Grid.RowDefinitions>

    <TextBlock FontSize="20" FontWeight="Bold" Grid.ColumnSpan="3"
        Grid.Row="0">Productos Comprados</TextBlock>
    <TextBlock FontSize="12" FontWeight="Bold" Grid.Row="1"
        Grid.Column="0">Enero</TextBlock>
    <TextBlock FontSize="12" FontWeight="Bold" Grid.Row="1"
        Grid.Column="1">Febrero</TextBlock>
    <TextBlock FontSize="12" FontWeight="Bold" Grid.Row="1"
        Grid.Column="2">Marzo</TextBlock>

```

```

<TextBlock Grid.Row="2" Grid.Column="0" TextAlignment="Right"
>50,000</TextBlock>
<TextBlock Grid.Row="2" Grid.Column="1"
    TextAlignment="Right">100,000</TextBlock>
<TextBlock Grid.Row="2" Grid.Column="2"
    TextAlignment="Right">150,000</TextBlock>
<TextBlock FontSize="16" FontWeight="Bold" Grid.ColumnSpan="3"
    Grid.Row="3">Total Units: 300,000</TextBlock>
</Grid>

```

Productos Comprados		
Enero	Febrero	Marzo
50,000	100,000	150,000
Total Units: 300,000		

StackPanel

Organiza los elementos secundarios en una única línea que se puede orientar horizontal o verticalmente. La propiedad *Orientation* permite organizar el contenido horizontal o verticalmente.

En el siguiente ejemplo se muestra el uso del control StackPanel. La propiedad *Orientation* organiza los RadioButtons en forma vertical.

```

<StackPanel Height="40" Width="115" Orientation="Vertical" >
    <RadioButton Name="rb1" Checked="rb_Checked" >Si</RadioButton>
    <RadioButton Name="rb2" Checked="rb_Checked" >No</RadioButton>
</StackPanel>

```



GroupBox

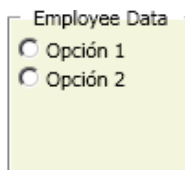
Representa un control que crea un contenedor que tiene un borde y un encabezado para el contenido de interfaz de usuario. Su propiedad de contenido es *Content* y su propiedad de encabezado es *Header*.

En el siguiente ejemplo se muestra el uso del control GroupBox. La propiedad *Header* permite ponerle el título. El uso de un *StackPanel*, otro de los controles contenedores, permite organizar el cuerpo del *GroupBox*.

```

<GroupBox Width="100" Height="100" Background="Beige" >
    <GroupBox.Header>
        <Label>Employee Data</Label>
    </GroupBox.Header>
    <StackPanel>
        <RadioButton HorizontalAlignment="Left" Height="16" Name="Opcion1"
            Width="120">Opción 1</RadioButton>
        <RadioButton HorizontalAlignment="Left" Height="16" Name="Opcion2"
            Width="120">Opción 2</RadioButton>
    </StackPanel>
</GroupBox>

```

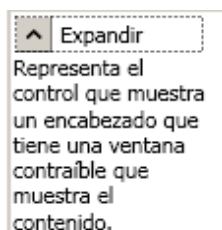


Expander

Representa el control que muestra un encabezado que tiene una ventana contraíble que muestra el contenido. Su propiedad de contenido es *Content* y su propiedad de encabezado es *Header*. Si el contenido de la ventana expandida es demasiado grande para la ventana, puede ajustar el contenido de *Expander* en un control *ScrollView* para proporcionar el contenido desplazable. *Expander* no proporciona automáticamente la función de desplazamiento.

En el siguiente ejemplo se muestra el uso del control *Expander*. Las propiedades de este control permiten controlar la expansión y contracción del cuerpo del mismo.

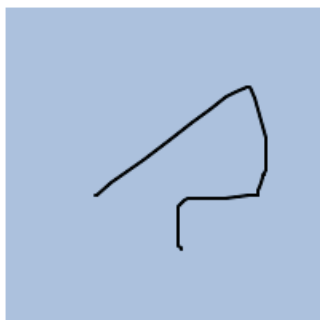
```
<Expander Name="myExpander" HorizontalAlignment="Left" Header="Expandir"
ExpandDirection="Down" IsExpanded="True" Width="100">
    <TextBlock TextWrapping="Wrap">
        Representa el control que muestra un encabezado que tiene una ventana
        contraíble que muestra el contenido.
    </TextBlock>
</Expander>
```



InkCanvas

InkCanvas es un elemento que se puede utilizar para recibir y mostrar entradas de entrada manuscrita. Normalmente esta operación se lleva a cabo mediante el uso de un lápiz o un mouse que interactúan con un digitalizador para generar trazos de entrada manuscrita. Los trazos creados se representan como objetos *Stroke* y se pueden manipular mediante programación o a través de los datos proporcionados por el usuario. *InkCanvas* permite al usuario modificar o eliminar un objeto *Stroke* existente.

```
<InkCanvas Name="ic" Background="LightSteelBlue" Canvas.Top="0" Canvas.Left="0"
Height="200" Width="200" />
```



Menu

Representa un control de menú de Windows que le permite organizar jerárquicamente los elementos asociados a comandos y controladores de eventos.

Presenta una lista de elementos que especifican comandos u opciones para una aplicación. Normalmente, al hacer clic en un elemento de un menú, se abre un submenú o una aplicación ejecuta un comando. *MenuItem* es el tipo más común de elemento de Menu. *MenuItem* puede contener elementos secundarios.

En el siguiente ejemplo se muestra el uso del control Menu y la forma de crear los MenuItem.

```
<Menu Name="menu1" Width="300" HorizontalAlignment="Left"
  VerticalAlignment="Top" >
  <MenuItem Header="_Archivo" >
    <MenuItem Header="A_brir" IsCheckable="true" />
    <MenuItem Header="_Cerrar" IsCheckable="true" IsChecked="True" />
    <MenuItem Header="_Guardar" >
      <MenuItem.Icon>
        <Image Source="setup.ico" Width="16" Height="16" />
      </MenuItem.Icon>
    </MenuItem>
    <Separator />
    <MenuItem Header="A_gregar" >
      <MenuItem Header="Proyecto" />
      <MenuItem Header="Archivo" />
    </MenuItem>
  </MenuItem>
</Menu>
```

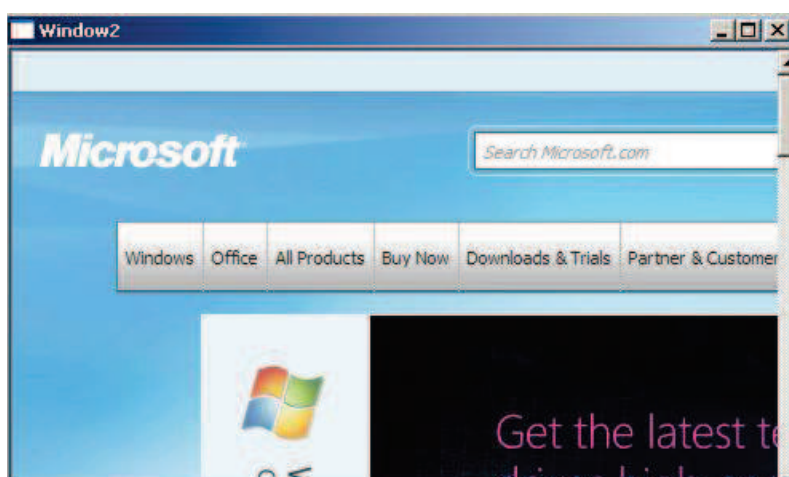


Frame

Es un control de contenido que proporciona la capacidad para navegar y mostrar el contenido. Se puede hospedar dentro de otro contenido, como con otros controles y elementos. El

contenido puede ser cualquier tipo de objeto de .NET Framework y archivos HTML. En general, sin embargo, las páginas son la manera preferida de empaquetar contenido para la navegación. A un contenido se puede navegar estableciendo la propiedad *Source* con el URI para el contenido deseado. Además, a un contenido se puede navegar utilizando una de las sobrecargas del método *Navigate*.

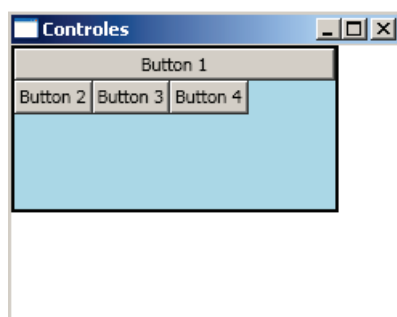
```
<Frame Name="miFrame" Source="http://www.microsoft.com" />
```



WrapPanel

Posiciona los elementos hijos en forma secuencial de izquierda a derecha, cortando el contenido de manera que quede sobre la línea siguiente al llegar al borde del contenedor. El orden se mantiene de arriba hacia abajo o de derecha a izquierda dependiendo del valor de la propiedad *Orientation*.

```
<Border HorizontalAlignment="Left" VerticalAlignment="Top" BorderBrush="Black"
BorderThickness="2">
    <WrapPanel Background="LightBlue" Width="200" Height="100">
        <Button Width="200">Button 1</Button>
        <Button>Button 2</Button>
        <Button>Button 3</Button>
        <Button>Button 4</Button>
    </WrapPanel>
</Border>
```



3- Otros Controles

Label - AccessText

Representa la etiqueta de texto de un control. Esta clase proporciona compatibilidad funcional y visual con las teclas de acceso. Se utiliza con frecuencia para permitir el acceso rápido desde el teclado a controles como *TextBox*. Para asociar un control *Label* a otro control, establezca la propiedad *Target* en el control que debe obtener el foco cuando el usuario presione la tecla de acceso. Para establecer la tecla de acceso, agregue un guión bajo antes del carácter que debe ser la tecla de acceso. Se accede con Alt + la letra seleccionada.

En el siguiente ejemplo al presionar Alt+D, el foco se posiciona en TextBox2

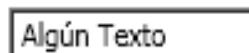
```
<DockPanel Margin="0,10,3,3" Grid.Column="0" Grid.Row="5">
  <TextBox Name="textBox2" Width="100" Height="20"/>
  <Label Target="{Binding ElementName=textBox2}">
    <AccessText FontSize="12" FontWeight="Bold">E_ditar</AccessText>
  </Label>
</DockPanel>
<TextBox Name="textBox3" Width="100" Height="20" Canvas.Top="30" />
```



TextBox

Representa un control que se puede utilizar para mostrar o editar texto sin formato.

```
<TextBox Name="tbSelectSomeText" Width="100" >
  Algún Texto</TextBox>
```



TextBlock

Proporciona un control ligero que permite mostrar pequeños fragmentos de contenido dinámico. TextBlock se ha diseñado para que sea ligero y se ha preparado específicamente para integrar pequeños fragmentos de contenido dinámico en una interfaz de usuario (UI). TextBlock proporciona un rendimiento óptimo en la presentación de líneas únicas y un rendimiento apropiado en la presentación de hasta unas pocas líneas de contenido.

```
<TextBlock Name="textBlock1" TextWrapping="Wrap" FontSize="14">
  <Bold>TextBlock</Bold> esta diseñado para ser <Italic>liviano</Italic>
</TextBlock>
```

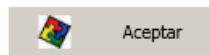
TextBlock esta diseñado para ser *liviano*

Button

Su propiedad de contenido es *Content*. Su evento principal es *Click*.

En el siguiente ejemplo se muestra un botón que contiene una imagen y un texto formado por controles contenidos.

```
<Button Name="btn5" Canvas.Left="64" Canvas.Top="17" Width="129" Height="33"
Click="btnAceptar_Click" >
  <StackPanel Orientation="Horizontal" Width="Auto">
    <Image Source="ms.jpg" Width="20" Height="20"></Image>
    <TextBlock Text="Aceptar" TextAlignment="Right" VerticalAlignment="Center"
      Width="69"></TextBlock>
  </StackPanel>
</Button>
```



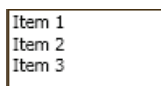
ListBox - ComboBox

El *ListBox* permite mostrar una lista de ítems desplegada. Las propiedades de contenido son las colecciones *Items* e *ItemsSource*. La propiedad *SelectionMode* permite establecer cuantos ítems pueden ser seleccionados. Las opciones son Single (1 solo ítem), Multiple (varios ítems) y Extended (se pueden seleccionar varios ítems consecutivos usando la tecla Shift o no consecutivos usando la tecla Ctrl).

El *ComboBox* permite ocultar y desplegar la lista de ítems. Las propiedades *IsEditable* e *IsReadOnly* especifican cómo se comporta ComboBox cuando el usuario escribe una cadena para seleccionar un elemento o copia y pega un valor en el cuadro de texto.

En el siguiente ejemplo vemos el uso de un ListBox sencillo.

```
<ListBox Name="lb" Width="100" Height="55" SelectionChanged="PrintText"
SelectionMode="Single">
  <ListBoxItem>Item 1</ListBoxItem>
  <ListBoxItem>Item 2</ListBoxItem>
  <ListBoxItem>Item 3</ListBoxItem>
</ListBox>
```



Estos controles permiten también el uso de listas con contengan CheckBox, imágenes, etc. como controles contenidos para cada ítem.

```
<ListBox Name="lb" Width="251" Height="119" SelectionMode="Single"
SelectionChanged="lb_SelectionChanged">
  <ListBoxItem Background="LightCoral" FontFamily="Verdana" FontSize="12"
    FontWeight="Bold">
    <CheckBox Name="chk1">
      <StackPanel Orientation="Horizontal">
        <Image Source="ms.jpg" Height="30"></Image>
        <TextBlock Text="Item 1" Width="174"></TextBlock>
      </StackPanel>
    </CheckBox>
  </ListBoxItem>
  <ListBoxItem Background="Green" FontFamily="Verdana" FontSize="12"
    FontWeight="Bold">
    <CheckBox Name="chk2">
```

```
<StackPanel Orientation="Horizontal">
    <Image Source="nextover.jpg" Height="30"></Image>
    <TextBlock Text="Item 2" Width="174"></TextBlock>
</StackPanel>
</CheckBox>
</ListBoxItem>
</ListBox>
```



```
<ComboBox Height="25" Width="150" HorizontalAlignment="Center"
VerticalAlignment="Top" IsEditable="True">
    <CheckBox Name="Item1">Buenos Aires</CheckBox>
    <CheckBox Name="Item2">Córdoba</CheckBox>
    <CheckBox Name="Item3">Mendoza</CheckBox>
</ComboBox>
```



CheckBox

Representa un control que un usuario puede activar y desactivar. Pueden tener tres estados: activado, desactivado e indeterminado.

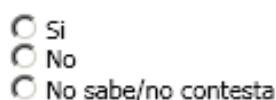
```
<CheckBox Name="Prueba" IsChecked="True">Prueba</CheckBox>
```



RadioButton

Representa un botón que el usuario puede seleccionar, pero no borrar. La propiedad *IsChecked* de *RadioButton* se puede establecer haciendo clic en él, pero sólo se puede borrar mediante programación. Normalmente se lo utiliza como elemento de un grupo de controles *RadioButton*. La selección está determinada por el estado de su propiedad *IsChecked*. Su evento principal es *Checked*.

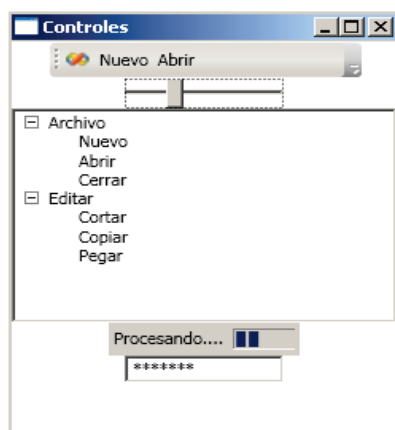
```
<StackPanel Height="40" Canvas.Left="35" Canvas.Top="43" Width="115">
    <RadioButton Name="rb1" Checked="rb_Checked">Si</RadioButton>
    <RadioButton Name="rb2" Checked="rb_Checked">No</RadioButton>
    <RadioButton Name="rb3" Checked="rb_Checked">No sabe/no
    contesta</RadioButton>
</StackPanel>
```



Otros Controles

- **ToolBar**: Identifica una barra de herramientas, como el control que contiene un conjunto de botones de comando en una ventana de la aplicación.
- **Slider**: Identifica un control deslizante.
- **TreeView**: Muestra datos jerárquicos, como una tabla de contenido, en una estructura de árbol.
- **StatusBar**: Identifica un control de barra de estado.
- **ProgressBar**: Identifica un control de barra de progreso, que indica visualmente el progreso de una operación prolongada.
- **PasswordBox**: Representa un control diseñado para escribir y administrar las contraseñas.

```
<StackPanel>
  <ToolBar Height="26" Name="toolBar1" Width="200" >
    <Button>
      <Image Source="setup.ico"/>
    </Button>
    <Button>Nuevo</Button>
    <Button>Abrir</Button>
  </ToolBar>
  <Slider Height="21" Name="slider1" Width="100" />
  <TreeView Height="150">
    <TreeViewItem Header="Archivo">
      <TreeViewItem Header="Nuevo" />
      <TreeViewItem Header="Abrir" />
      <TreeViewItem Header="Cerrar" />
    </TreeViewItem>
    <TreeViewItem Header="Editar">
      <TreeViewItem Header="Cortar" />
      <TreeViewItem Header="Copiar" />
      <TreeViewItem Header="Pegar" />
    </TreeViewItem>
  </TreeView>
  <StatusBar Height="23" Name="statusBar1" Width="120" >
    <TextBlock Height="15" Text="Procesando...." />
    <ProgressBar Height="15" Name="progressBar1" Width="100" Value="20" />
  </StatusBar>
  <PasswordBox Name="pwdBox" Width="100" MaxLength="64" PasswordChar="*" />
</StackPanel>
```



4- Información General de Controles

Propiedades Generales de los Controles

BitmapEffect: Obtiene o establece efectos visuales.

```
<Button.BitmapEffect>
    <BevelBitmapEffect />
</Button.BitmapEffect>
```



```
<Button.BitmapEffect>
    <DropShadowBitmapEffect />
</Button.BitmapEffect>
```



Focusable: Obtiene o establece un valor que indica si el foco se puede establecer en este elemento. Para exigir que un elemento del panel reciba el foco, establezca la propiedad *Focusable* en true.

HorizontalAlignment: Obtiene o establece las características de alineación horizontal que se aplican a este elemento cuando se crea dentro de un elemento primario, como un panel o control de elementos.

IsEnabled: Obtiene o establece un valor que indica si este elemento está habilitado en la interfase de usuario.

Margin: Permite establecer los márgenes de un control con respecto a su contenedor. Se escribe en el siguiente formato (izquierda, arriba, derecha y abajo). Ejemplo: `Margin="50,10,50,10"`

MaxHeight: Obtiene o establece la restricción de alto máximo del elemento.

MaxWidth: Obtiene o establece la restricción de ancho máximo del elemento.

MinHeight: Obtiene o establece la restricción de alto mínimo del elemento.

MinWidth: Obtiene o establece la restricción de ancho mínimo del elemento.

Opacity: Permite generar objetos transparentes o semi transparentes. Los valores aceptados van del 0 al 1, siendo 0 totalmente transparente y 1 totalmente opaco.

Padding: Obtiene o establece el relleno interior del control. El relleno se establece en el orden siguiente: izquierda, arriba, derecha y abajo. El valor predeterminado es un grosor de 0 en los cuatro lados.

VerticalAlignment: Obtiene o establece las características de alineación vertical que se aplican a este elemento cuando se crea dentro de un elemento primario, como un panel o control de elementos.

Visibility: Obtiene o establece la visibilidad de la interfaz de usuario de este elemento.

Creación dinámica de controles

Todos los controles pueden ser también generados dinámicamente en tiempo de ejecución.

```
// Crea la ventana principal de la aplicación
mainWindow = new Window();
mainWindow.Title = "Grid Sample";
// Crea el control Grid
Grid myGrid = new Grid();
myGrid.Width = 250;
myGrid.Height = 100;
```



```
myGrid.HorizontalAlignment = HorizontalAlignment.Left;
myGrid.VerticalAlignment = VerticalAlignment.Top;
myGrid.ShowGridLines = true;
// Define la columnas
ColumnDefinition colDef1 = new ColumnDefinition();
ColumnDefinition colDef2 = new ColumnDefinition();
ColumnDefinition colDef3 = new ColumnDefinition();
myGrid.ColumnDefinitions.Add(colDef1);
myGrid.ColumnDefinitions.Add(colDef2);
myGrid.ColumnDefinitions.Add(colDef3);
// Define las filas
RowDefinition rowDef1 = new RowDefinition();
RowDefinition rowDef2 = new RowDefinition();
RowDefinition rowDef3 = new RowDefinition();
RowDefinition rowDef4 = new RowDefinition();
myGrid.RowDefinitions.Add(rowDef1);
myGrid.RowDefinitions.Add(rowDef2);
myGrid.RowDefinitions.Add(rowDef3);
myGrid.RowDefinitions.Add(rowDef4);

// Agrega el texto a la primera celda
TextBlock txt1 = new TextBlock();
txt1.Text = "Productos Comprados";
txt1.FontSize = 20;
txt1.FontWeight = FontWeights.Bold;
Grid.SetColumnSpan(txt1, 3);
Grid.SetRow(txt1, 0);
.....
```

